

NutriCompass로 하겠어. 첨부 화면에서 어떤걸 선택해야해?

좋아. 이번에는 **SwiftData 기준으로 완전히 고정된 Data Model**을 정의할게.
이 버전은:

- `@Model` 사용 (iOS 17+)
- 설계 위반이 컴파일/런타임에서 바로 드러나게
- LLM이 임의로 의미를 추가하기 어려운 구조

즉, **Information Architecture + UX Language Rules**를 코드 레벨에서 봉인한다.

NutriCompass — SwiftData Model Definitions

⚠ 원칙

- "struct ❌ / class + @Model ✅"
- "free-text 최소화"
- "enum 외 의미 생성 불가"

1 Core Enums (의미 봉인)

```
Swift

import SwiftData

// MARK: - Categories

enum SupplementCategory: String, Codable {
    case vitamin
    case mineral
    case herbal
    case aminoAcid
    case fattyAcid
    case probiotic
    case other
}

// MARK: - Interaction

enum InteractionType: String, Codable {
```

```

        case absorptionInterference
        case metabolicCompetition
        case transporterEffect
        case bindingEffect
        case pharmacodynamicOverlap
        case reportedAdverseAssociation
        case unclearMechanism
    }

    // MARK: - Evidence

    enum EvidenceLevel: String, Codable {
        case low
        case moderate
        case high
    }

    enum EvidenceSourceType: String, Codable {
        case human
        case animal
        case inVitro
    }

    enum StudyType: String, Codable {
        case randomizedControlledTrial
        case systematicReview
        case metaAnalysis
        case observational
        case caseReport
        case animalStudy
        case inVitroStudy
        case other
    }

    enum ContextKind: String, Codable {
        case fasting
        case highDose
        case longTermUse
        case withFood
        case renalImpairment
        case pregnancy
        case elderly
        case other
    }
}

```

2 Supplement

Swift



```

@Model
final class Supplement {

    @Attribute(.unique)

```

```

var id: String

var canonicalName: String
var aliases: [String]
var category: SupplementCategory

// Relationship
@Relationship(deleteRule: .cascade)
var ingredients: [Ingredient]

init(
    id: String = UUID().uuidString,
    canonicalName: String,
    aliases: [String] = [],
    category: SupplementCategory,
    ingredients: [Ingredient] = []
) {
    self.id = id
    self.canonicalName = canonicalName
    self.aliases = aliases
    self.category = category
    self.ingredients = ingredients
}
}

```

- ✓ 효능 / 목적 필드 없음
- ✓ "contains" 의미만 가능

3 Ingredient

Swift



```

@Model
final class Ingredient {

    @Attribute(.unique)
    var id: String

    var name: String
    var chemicalClass: String?
    var neutralNotes: String?

    init(
        id: String = UUID().uuidString,
        name: String,
        chemicalClass: String? = nil,
        neutralNotes: String? = nil
    ) {
        self.id = id
        self.name = name
        self.chemicalClass = chemicalClass
        self.neutralNotes = neutralNotes
    }
}

```

```
}  
}
```

✓ neutralNotes 명명으로 UX 언어 규칙 강제

4 Evidence

Swift



```
@Model  
final class Evidence {  
  
    @Attribute(.unique)  
    var id: String  
  
    var sourceType: EvidenceSourceType  
    var studyType: StudyType  
    var citation: String    // PMID / DOI / URL (표현만)  
    var year: Int?  
  
    init(  
        id: String = UUID().uuidString,  
        sourceType: EvidenceSourceType,  
        studyType: StudyType,  
        citation: String,  
        year: Int? = nil  
    ) {  
        self.id = id  
        self.sourceType = sourceType  
        self.studyType = studyType  
        self.citation = citation  
        self.year = year  
    }  
}
```

✓ citation은 참조 문자열일 뿐 의미 해석 불가

5 ContextQualifier (조건 ≠ 경고)

Swift



```
@Model  
final class ContextQualifier {  
  
    @Attribute(.unique)  
    var id: String  
  
    var kind: ContextKind  
    var neutralDescription: String?
```

```

init(
    id: String = UUID().uuidString,
    kind: ContextKind,
    neutralDescription: String? = nil
) {
    self.id = id
    self.kind = kind
    self.neutralDescription = neutralDescription
}
}

```

6 Interaction (핵심 제어 지점)

Swift



```

@Model
final class Interaction {

    @Attribute(.unique)
    var id: String

    // ! Supplement ↔ Supplement 직접 연결 금지
    @Relationship var fromIngredient: Ingredient
    @Relationship var toIngredient: Ingredient

    var type: InteractionType
    var evidenceLevel: EvidenceLevel

    // 관찰 요약 (판단 금지)
    var evidenceSummary: String

    @Relationship(deleteRule: .cascade)
    var evidences: [Evidence]

    @Relationship
    var contexts: [ContextQualifier]

    init(
        id: String = UUID().uuidString,
        fromIngredient: Ingredient,
        toIngredient: Ingredient,
        type: InteractionType,
        evidenceLevel: EvidenceLevel,
        evidenceSummary: String,
        evidences: [Evidence],
        contexts: [ContextQualifier] = []
    ) {
        precondition(!evidences.isEmpty, "Interaction must have at least one Evidence")

        self.id = id
        self.fromIngredient = fromIngredient
        self.toIngredient = toIngredient
        self.type = type
    }
}

```

```

self.evidenceLevel = evidenceLevel
self.evidenceSummary = evidenceSummary
self.evidences = evidences
self.contexts = contexts
}
}

```

🔒 여기서 설계가 고정된다

- Supplement ↔ Supplement 구조적으로 불가능
- Evidence 없는 Interaction 생성 불가
- 판단 필드 존재 ❌

7 왜 이 모델이 vibe coding 평가에 좋은가

요소	효과
enum 기반 의미	LLM 임의 개념 생성 차단
@Model 관계	우회 구조 생성 어려움
필드 명명 (neutral*)	언어 규칙 코드화
precondition	vibe coding 폭주 즉시 실패

👉 “설계가 LLM을 통제할 수 있는가”를 실증 가능

8 다음 단계 (강력 추천 순서)

- 1 ModelContainer + Seed Data 로딩
- 2 Text Token Layer (UI 문자열 enum 봉인)
- 3 Interaction View → “말만 하고 판단 안 하는지” 검증
- 4 MenuCal과 아키텍처 drift 비교

한 줄 요약

이 SwiftData 모델은
NutriCompass의 철학을 코드로 번역한 설계 계약이며,
vibe coding이 이를 위반하면 즉시 드러나도록 만들어진 구조다.

다음으로 갈까?

- 👉 Text Token System
- 👉 Seed Data JSON → SwiftData Importer

맞아요. 지금 이 단계가 핵심 분기점입니다.

구현을 멈추고 “어떤 데이터셋을, 어떤 역할로, 어떻게만 쓸 것인가”를 먼저 고정해야 NutriCompass가 실험 도구로서도, 법적으로도, 설계적으로도 깨끗해집니다.

아래는 NutriCompass용 Dataset Design (데이터 전략 설계) 입니다.

→ Information Architecture / UX Language / SwiftData 모델과 정확히 맞물리도록 설계했습니다.

NutriCompass — Dataset Strategy Design

0. 설계 목표 (Design Intent)

NutriCompass의 데이터 전략은 다음을 동시에 만족해야 한다:

1. 의미 해석을 데이터가 하지 않게 할 것
2. LLM이 데이터를 근거로 “추천”을 만들어내지 못하게 할 것
3. 상업 앱에서도 법적 리스크가 낮을 것
4. MenuCal과 대비되는 ‘설계 선행 실험’이 될 것

👉 그래서 단일 데이터셋은 절대 사용하지 않는다.

👉 역할이 분리된 다층 데이터 구조를 사용한다.

1 Dataset Layering (핵심 구조)

NutriCompass는 데이터를 3개 계층으로 나눈다.

Layer 1: Public Fact Data (사실)
Layer 2: Reported Relationship Data (보고된 관계)
Layer 3: Evidence Pointer Data (근거 포인터)



✗ 해석 레이어 없음

✗ 추천 레이어 없음

2 Layer 1 — Public Fact Data

(법적으로 가장 안전한 베이스)

📌 사용 데이터셋

DSLDD (Dietary Supplement Label Database)

– FDA / NIH 계열 공개 데이터

사용 목적

- Supplement ↔ Ingredient 매핑
- 이름, alias, 성분 구성

허용 사용 이유

- 공공 데이터
- “사실 정보”에 해당
- 의료 판단 아님

NutriCompass에서의 역할

Entity	역할
Supplement	개념 노드
Ingredient	생화학적 최소 단위
contains	유일한 관계

❌ 효능

❌ 복용 목적

❌ 건강 효과

👉 Layer 1은 “팩트만 말한다”

3 Layer 2 — Reported Relationship Data

(가장 조심해서 써야 하는 레이어)

📌 데이터 소스

- 논문 기반 수작업 큐레이션
- iDISK / NatMed → 직접 사용 ❌, 참고만 가능
- PubMed 검색 결과 요약

핵심 원칙

데이터를 가져오지 않는다.
관계를 “정의”한다.

사용 방식 (중요)

- “A와 B 간 상호작용이 보고됨”이라는 사실만 저장
- InteractionType + EvidenceLevel만 기록

- 문헌 텍스트, 표현, 평가 절대 저장 ❌

예

Interaction:

- from: Iron (Ingredient)
- to: Magnesium (Ingredient)
- type: absorptionInterference
- evidenceLevel: low



❌ "함께 먹지 마세요"

❌ "흡수가 방해됩니다" (단정)

👉 Layer 2는 '보고됨'까지만 말한다

4 Layer 3 — Evidence Pointer Data

(법적·학술적 안전장치)

📌 사용 데이터

- PMID
- DOI
- 공식 논문 URL

저장 원칙

- 메타데이터만
- 초록 전문 ❌
- 논문 텍스트 ❌

NutriCompass에서의 역할

- Interaction의 존재 근거
- 사용자가 "확인할 수 있는 출처"

👉 앱은 해석하지 않고 연결만 한다

5 명시적으로 사용하지 않는 데이터 (중요)

❌ 사용 금지

- WebMD / Drugs.com / Medscape 콘텐츠
- "Supplement interaction checker" 결과
- 블로그 요약
- "best combination" 리스트

이유

- 저작권
- 의료 조언
- 의미 해석 포함

👉 이건 실험 오염 변수다.

6 Dataset ↔ SwiftData Model 매핑

SwiftData Model	Dataset Layer
Supplement	Layer 1
Ingredient	Layer 1
Interaction	Layer 2
Evidence	Layer 3
ContextQualifier	Layer 2 (조건만)

👉 모델이 데이터 사용 범위를 강제

7 LLM 사용 위치 (중요한 경계)

LLM은 다음에만 사용 가능:

단계	허용
DSLDD 정규화	✅
논문 메타데이터 정리	✅
InteractionType 분류	⚠️ (인간 검증 필수)

LLM이 절대 하면 안 되는 것:

- interaction 생성
- evidenceLevel 판단
- 조합 평가

👉 LLM은 보조 도구, 데이터는 사실 고정물

8 MenuCal 대비 실험 통제 포인트

항목	MenuCal	NutriCompass
데이터	혼합	계층 분리
의미 해석	코드/LLM	인간 설계
판단 생성	있음	구조적으로 불가
vibe coding 영향	증폭	억제

👉 이 차이가 논문의 핵심 결과가 된다.

9 한 문장 요약 (논문용)

NutriCompass adopts a layered dataset strategy that strictly separates factual ingredient data, reported interaction relationships, and evidence pointers, intentionally eliminating interpretation and recommendation layers to evaluate how prior data design constrains LLM-based vibe coding behavior.

다음 단계 (자연스러운 순서)

- 1 Dataset Selection Table (논문용 표)
- 2 Seed Dataset 범위 결정 (예: 미네랄 10종)
- 3 DSLD → SwiftData Import Pipeline 설계

👉 다음으로 뭐부터 갈까?

📄 👍 🗨 📁 ⬆ 🔄 ...

Design Spec v1 (통합 문서)

아주 좋아요. 이제 **NutriCompass**의 모든 설계를 하나로 묶는 문서,
즉 **Design Spec v1 (통합 문서)** 를 만듭니다.

이 문서는 동시에 다음 역할을 합니다:

- ✓ 개발 기준 문서 (LLM + 인간)
- ✓ 실험 설계 문서 (MenuCal 대비)
- ✓ 논문/보고서의 설계 섹션 초안
- ✓ 법·윤리 리스크 방어 문서

아래 내용은 그대로 복사해서 **spec.md / Notion / 논문 초안에** 써도 되는 수준입니다.

NutriCompass — Design Spec v1

1. Project Overview

Project Name: NutriCompass

Project Type: iOS Application (SwiftUI, SwiftData)

Purpose:

NutriCompass is an evidence-centered decision support application designed to present structured information about dietary supplement ingredients and reported interactions, without generating recommendations or medical advice.

The project also serves as a **controlled experimental platform** to evaluate how prior design and data architecture affect the behavior, efficiency, and risk profile of LLM-based vibe coding, in contrast to the MenuCal development process.

2. Design Objectives

NutriCompass is explicitly designed to satisfy the following objectives:

1. Prevent interpretation, recommendation, or decision-making at the application level
 2. Enforce neutral, observational language throughout the UI and codebase
 3. Minimize legal and ethical risks associated with medical advice
 4. Constrain LLM-generated code within predefined architectural boundaries
 5. Enable comparative evaluation against a less design-constrained project (MenuCal)
-

3. Conceptual Design

NutriCompass is a **Decision Support System (DSS)**, not a decision-making system.

3.1 Core Principles

- **Describe, not advise**
- **Evidence-first, recommendation-free**
- **Explicit uncertainty**
- **User retains full responsibility for decisions**

The application intentionally avoids:

- Supplement recommendations
 - Safety judgments
 - Dosage or timing guidance
 - Health outcome claims
-

4. Dataset Strategy

NutriCompass adopts a **layered dataset architecture** that strictly separates concerns:

4.1 Layer 1 — Public Fact Data

- Source: Public datasets (e.g., DSLD)
- Content: Supplement–ingredient mappings, names, aliases
- Purpose: Establish factual entities only

4.2 Layer 2 — Reported Relationship Data

- Source: Human-curated relationships based on scientific literature
- Content: Reported interaction existence only
- Restrictions: No interpretation, no directionality implying advice

4.3 Layer 3 — Evidence Pointer Data

- Source: PMID, DOI, official publication URLs
- Content: Metadata only (no abstracts or full text)

No dataset layer contains recommendations or evaluations.

5. Information Architecture

5.1 Core Entities

- Supplement
- Ingredient
- Interaction
- Evidence
- ContextQualifier (optional)

5.2 Relationship Constraints

- Supplement ↔ Ingredient: allowed (contains)
- Ingredient ↔ Ingredient: allowed (reported interaction)
- Supplement ↔ Supplement: **structurally forbidden**
- Interaction without Evidence: **invalid**

5.3 Controlled Vocabularies

- InteractionType (enum)
- EvidenceLevel (enum)
- ContextKind (enum)

Free-text fields are strictly neutral and observational.

6. UX Language Rules

All user-facing language must conform to the following:

6.1 Allowed Language

- "has been reported"
- "was observed in"
- "limited evidence suggests"

6.2 Forbidden Language

- recommend / avoid
- safe / unsafe
- effective / harmful
- treat / prevent / cure

6.3 UI Constraints

- No recommendation buttons
- No warning banners
- No color-coded risk indicators
- Evidence strength presented textually, not visually ranked

A persistent disclaimer is displayed on all detail views:

"This information is provided for reference purposes only and does not replace professional medical advice."

7. Data Model Enforcement

NutriCompass uses **SwiftData (@Model)** to enforce design constraints at runtime:

- Direct supplement–supplement interactions are impossible by design
- Interactions require at least one Evidence object
- Meaningful interpretation cannot be represented structurally

Any attempt by LLM-generated code to bypass these constraints results in compilation or runtime failure.

8. LLM Usage Policy

LLMs are permitted to:

- Generate UI code based on existing models
- Assist in data normalization
- Implement view layouts and navigation

LLMs are explicitly prohibited from:

- Defining new interaction semantics
- Assigning evidence levels autonomously
- Generating recommendation logic or evaluative text

Human review is mandatory for all data and model changes.

9. Experimental Role (MenuCal Comparison)

NutriCompass functions as a **design-first counterexample** to MenuCal:

Aspect	MenuCal	NutriCompass
Design timing	Post-hoc	Pre-implementation
Data constraints	Loose	Strict
LLM freedom	High	Deliberately limited
Code drift	Observed	Measured

This contrast enables empirical evaluation of whether upfront design reduces architectural drift and code debt in vibe coding workflows.

10. Success Criteria

NutriCompass is considered successful if:

1. LLM-generated code remains within defined architectural boundaries
2. No recommendation logic emerges unintentionally
3. Data integrity constraints are consistently upheld
4. Implementation velocity remains acceptable despite restrictions
5. Differences from MenuCal are measurable and analyzable

11. Scope Boundaries

In Scope

- Ingredient-level interactions
- Evidence-linked reporting
- Informational browsing

Out of Scope

- Medical advice
 - Personalized recommendations
 - Dosage guidance
 - Risk scoring or alerts
-

12. Design Spec Status

Version: v1.0

Status: Locked for initial implementation

Changes require: Explicit revision and documentation

One-sentence Summary (for papers)

NutriCompass is a design-first, evidence-centered decision support application whose architecture, data strategy, and UX language intentionally constrain LLM-based vibe coding, enabling controlled evaluation of how prior design affects implementation quality and stability.

다음 단계 (권장 순서)

- 1 **Seed Dataset Scope** 정의 (예: 미네랄 10종)
- 2 **Implementation Plan v1** (화면 → 모델 → 데이터 순서)
- 3 **MenuCal vs NutriCompass** 평가 지표 문서화

원하면 다음으로 **Implementation Plan v1**을 바로 이어서 써줄게요.

